# Final Project

Build an AI agent app that does something useful for real people. Then show how it works, what it costs, and how you kept it safe.

Due **Monday, October 26, 2026 at 6:00 PM** on Canvas

## The big idea

All semester you built agents piece by piece: tools, structured outputs, chat websites, multi-agent teams, caching, and defenses against prompt injection. Now you put it together on a problem **you** care about.

Pick something that matters to someone: a business, a nonprofit, a classroom, a hospital, your old job, your family, your community. It does not have to make money. It has to make a difference. The best projects are ones you could hand over to a real person tomorrow and they would actually use it.

## What you submit

| What | Where | Details |
|---|---|---|
| **GitHub repo** | Canvas (website URL) | Submit the link to your repo. It holds all the code for your app, with a README that explains how to run it. Public, or private with **zlisto** added. See Repo. |
| **Report** (`report.html`) | Canvas | A single HTML file, 10–15 pages when printed. See Report. |

| | | |
|---|---|---|
| **Slide deck** (`slides.html`) | Canvas | A single HTML file that summarizes the project, 10 slides max. See Slides. |
| **Demo video** (optional) | Canvas | A 2–3 minute screen recording of you using the app. See Demo video. |

This is a **solo project**. On Canvas you submit three things: the repo link, `report.html`, and `slides.html`. You can also add an optional demo video.

**Self-contained HTML:** both HTML files should open by double-clicking them, on any computer, with every screenshot showing. Ask your vibe coder to embed the images inside the file so nothing is missing when the file is opened.

## The report (10–15 pages)

Write it for a smart manager who has never seen your app. Use these sections, in this order. Put screenshots of your app wherever they help the reader see what you are describing.

**You can write it with AI assistance.** Use your vibe coder or any AI tool to draft, edit, and build the HTML. You are responsible for every word and number in it, so check that it matches what your app actually does.

| Section | What goes in it |
|---|---|
| **1. Title + links** | Project name, your name, the GitHub repo link (same one you submit on Canvas), the link to the app itself if it is online, and one sentence that says what the app does. |
| **2. Motivation** | Who has this problem? How do they deal with it today, and what does that cost them in money, time, or frustration? Why should anyone care? |
| **3. What is new here** | Look around first. What already exists (apps, startups, ChatGPT itself)? What does your |

| | |
|---|---|
| | project do that those don't? Be honest. If something close exists, say what you do differently and why that matters. |
| **4. The agents** | • What are the AI agents in your app, and what does each one do?<br>• What tools can each agent use?<br>• If you have more than one agent: how are they organized? Who hands work to whom? Draw a diagram.<br>• Which model(s) you used and why.<br>• Walk through one real run, step by step, with screenshots. |
| **5. Tokenomics** | • How many tokens does one typical run use (input, cached input, output)? Measure it from real runs.<br>• What does one run cost? What does it cost per user per month?<br>• What would it cost at scale: 100 users? 10,000 users?<br>• What did you do to keep cost down?<br>• Did any of your cost cutting make the app perform worse? Show evidence. |
| **6. Safety** | What could go wrong, and what did you build to stop it? Cover the risks that matter for your app, for example:<br>• **Attacks:** prompt injection, users trying to get the agent to leak data or misbehave.<br>• **Agent malfunction:** the agent invents facts, takes the wrong action, or loops forever. How do you protect the user?<br>• **Overspending:** what stops a bug or a bad actor from burning through your token budget?<br>Show at least one defense actually working, with a screenshot or a test. |

| | |
|---|---|
| **7. Impact** | If this helps an organization or people, put numbers on it: money saved, hours saved, faster turnaround, better quality, more people served, or something that was impossible before and now is possible. Say where your numbers come from (your own tests, a pilot user, public data, a careful estimate). The strongest evidence is a real user trying it. |
| **8. Limits + next steps** | What doesn't work yet? What would you build next if you had another month? |

The page count is for a normal printed page (letter size, regular font, screenshots included). Fifteen pages of real detail beats fifteen pages of filler. The grade is for the content, and filler costs points.

## The slide deck

An HTML slide deck, **10 slides max**, that tells the story of your project in a few minutes. Someone flipping through it should understand the problem, what you built, and why it matters. A good order:

1. Title: project name, your name, one-line pitch
2. The problem and who has it
3. What is new about your solution
4. How the agents work (diagram)
5. Demo screenshots
6. Cost to run
7. Safety
8. Impact: the numbers
9. What's next

**Use reveal.js.** It is a free, popular package for making slide decks as a web page. You get one HTML file that opens in any browser, moves slide to slide with the arrow keys, and goes full screen when you press **F**. Your vibe coder already knows it, so you just ask. For example, type something like this in your own words:

> Make me a slide deck called slides.html using reveal.js, loaded from a CDN. 10 slides max, following the outline in my report. Put my screenshots and diagram right inside the file so it still works when emailed to someone. Use a clean design with big text and the same colors as my app.

Then double-click `slides.html` to open it and press the arrow keys to check every slide. Want ideas for the look? Flip through any of the lecture slide decks.

## The GitHub repo

- All the code someone needs to run your app.
- A `README.md` that says what the app is and how to run it, step by step (or the link, if it is online). See Testing the app.
- A `.env.example` that lists the keys needed, with no real values. **Never commit your real `.env` or your `PORTKEY_API_KEY`.**
- Submit the repo link on Canvas. Make the repo **public**, or if it is private, add the GitHub user **zlisto** as a collaborator (repo **Settings → Collaborators → Add people**) so it can be cloned.

### Model + budget

You have two choices:

- **Course key:** use your `PORTKEY_API_KEY` with **gpt-6-luna** only. No other models on the course key.
- **Your own key:** if you want a different model, use your own personal API key and pay for it yourself. For an OpenAI key, name it `OPENAI_API_KEY`.

## Testing the app

The teaching staff will actually use your app. Pick one of these two ways:

1. **Put it online.** Put the link at the top of your README and your report. Make sure it stays up and working until final grades are posted. Free hosting often goes to sleep, so check your link the day you submit.
2. **Make it run locally from a fresh clone.** The teaching staff will clone your repo and run it by following your README, step by step.

**Keep the local setup simple.** If your app needs anything unusual to run (special software, a big database, many services), put it online instead.

**The 10-minute rule:** if the app is not running after 10 minutes of following your README, it gets 0 points for "The teaching staff can run it."

### Where your app finds the API keys

Every repo gets cloned into one folder with all the other submissions. The teaching staff's `.env` file sits in that folder, one level **above** your repo:

```
submissions/
├── .env                ⟵ the teaching staff's keys live here
├── your-project/       ⟵ your repo, cloned
│   ├── README.md
│   ├── .env.example
│   └── ...
└── another-project/
```

So your app must look for the `.env` file in **two places**: first in your repo's own folder, then in the folder above it. Tell your vibe coder exactly that. Then test it yourself: move your `.env` up one folder and check that the app still runs.

The teaching staff's `.env` has two keys: `PORTKEY_API_KEY` and `OPENAI_API_KEY`. Your app must use one of those two names, spelled exactly like that. List every key your app needs in `.env.example`. If your app needs any other key, put the app online and consult the professor.

### Big data

If your app uses a large dataset, include a small **test dataset** in the repo so the app runs locally without it. Say in your README how to switch between the two.

### Demo video

**Optional, and a good backup.** Worried your app might not run on the teaching staff's computer for some reason, like data paths, keys, or setup? Record a **2–3 minute** screen recording of the app running on your own

computer, on a real example, start to finish. Talk through what is happening. Upload the video file on Canvas with the rest of your submission. If the app won't run for the teaching staff, the video can still earn the points for "It works" and "Look + feel." It cannot earn the 5 points for "The teaching staff can run it."

### Before you submit

Do a dry run: clone your repo into a brand new folder, put your `.env` one level above it, follow your own README, and use the app. If it breaks for you, it will break for the teaching staff.

## Grading (100 points)

There are three tiers. Every project lands in one of them.

Tier 1 **It works.** The app runs and does what it says. The idea is familiar (a chatbot on top of some documents, a version of something that already exists). The report has the sections, but skips details: few real numbers, thin on cost or safety. **This is a solid baseline grade.**

Tier 2 **It's clever and new.** Everything in Tier 1, plus an idea not seen before, or a smart new spin on an old problem. The agents are designed thoughtfully. The report is complete, with measured token costs, a real safety defense, and screenshots that show the app working.

Tier 3 **It makes a difference.** Everything in Tier 2, plus a very clever idea that clearly makes something better for real people, and could actually be used in the real world. You show evidence: a real user tried it, you measured time or quality against how it's done today, you thought through what it costs to run for a whole organization. This is not only about money. Helping a nonprofit serve more families or

helping a teacher reach every student counts just as much as saving a company a million dollars.

### Overdone ideas

Some ideas show up again and again. These are capped at **Tier 1 on the idea** unless you add a real twist that makes them new:

- Resume or cover letter reviewer
- Chat with a PDF or a pile of documents
- Trip or travel planner
- Email or meeting summarizer
- Recipe or meal planner
- Generic study buddy or flashcard maker
- Anything built in class or in a homework, without a real new addition

### Evidence earns full credit

Anyone can write a section with AI. Full credit goes to sections backed by proof from your real app. In each report row below, a section without the evidence listed gets **at most half** of that row's points.

Here is how the points break down.

| Component | Points | What earns the top of the range |
|---|---|---|
| **THE IDEA** | | **21 POINTS** |
| **Novelty + real-world value** | 21 | A very clever idea, clearly new, that solves a real problem for real people and could actually be used. This is where the tiers matter most. Overdone ideas without a real twist are capped at Tier 1. |
| **THE APP** | | **30 POINTS** |
| **It works** | 15 | The app does real work end to end: the agents use real tools, real data, and produce useful results, without crashing. Clean repo, no secrets committed. |
| **The teaching staff can run** | 5 | It works online, or it runs locally from a fresh clone by following the README, with |

| Component | Points | What earns the top of the range |
|---|---|---|
| **it** | | the `.env` one folder up. A test dataset is included if the real one is large. Not running after 10 minutes = 0. See Testing the app. |
| **Look + feel** | 10 | The app looks good and is easy to use: a clean layout, clear buttons and screens, and a person who has never seen it can figure it out. |
| **THE REPORT** | | **41 POINTS** |
| **Motivation + what is new** | 8 | A clear problem with a real person behind it. An honest look at what already exists and a sharp case for what is different. *Evidence: named existing products you compared against.* |
| **Agent design** | 10 | Each agent and tool explained, a clear diagram, smart choices about how agents are organized. *Evidence: one real run traced step by step with screenshots of what each agent did.* |
| **Tokenomics** | 8 | Costs per run, per user, and at scale. Concrete steps taken to cut cost, and whether they hurt performance. *Evidence: real token counts from your runs, shown in your logs or in screenshots of your app (your app can display its own token usage). Estimates alone get half.* |
| **Safety** | 8 | The risks that actually matter for this app, with defenses that are built. *Evidence: a screenshot or test of a defense actually stopping an attack or a malfunction. Describing a defense gets half.* |

| Component | Points | What earns the top of the range |
|---|---|---|
| **Impact + metrics** | 5 | Believable numbers for cost, time, quality, or reach, with sources. Honest limits and next steps. *Evidence: a real user tried it (a quote, screenshot, or before-and-after time). Guesses alone get half.* |
| **Design** | 2 | The report looks good: a clean layout, easy to read, with screenshots and diagrams that are sharp and well placed. |
| **THE SLIDES** | | **8 POINTS** |
| **Content** | 6 | Tells the whole story in a few minutes, with a diagram, screenshots, and the key numbers. |
| **Design** | 2 | The slides look good: big readable text, a consistent style, and visuals that carry the story. |
| **Total** | **100** | |

## Tips

- **Start with a person, then the tech.** "My aunt spends 6 hours a week on insurance paperwork for her clinic" is a better starting point than "Build a multi-agent system."
- **Talk to a real user.** Even one 15-minute conversation, or watching them try your app, makes Tier 3 much easier to reach.
- **Screenshot as you go.** Grab screenshots of good runs, bad runs, and your safety defenses catching something.

- **Ship something that works first.** A simple app that runs beats an ambitious one that crashes. Then make it clever.

© 2026 Tauhid Zaman